

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO

ĐỒ ÁN CHUYÊN NGÀNH
HỌC KỲ: 251 - NĂM HỌC: 2025-2026

HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ CHO BÁN LẺ
ĐIỆN THOẠI TÍCH HỢP TRỢ LÝ ẢO AI AGENT

NGÀNH: KHOA HỌC MÁY TÍNH

HỘI ĐỒNG:

GVHD: PGS. TS Quản Thành Thơ.

TKHD:

—o0o—

SVTH1: Phạm Quốc Toàn - 2213539

SVTH2: Nguyễn Ngọc Quế Chi - 2210371

SVTH3: Trần Nguyễn Anh Khoa - 2211651

TP. Hồ Chí Minh, Tháng 12/2025

Chữ ký giảng viên hướng dẫn

Ngày:

Phó giáo sư, Tiến sĩ Quản Thành Thơ (giảng viên hướng dẫn)
Khoa Khoa học và Kỹ thuật Máy tính

Lời cảm ơn

Nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến Phó Giáo sư, Tiến sĩ Quản Thành Thơ – người đã tận tình hướng dẫn, chỉ bảo và đồng hành cùng chúng em trong suốt quá trình thực hiện giai đoạn đề án chuyên ngành. Những định hướng quý báu, những góp ý xác đáng cùng sự hỗ trợ tận tâm của Thầy đã giúp chúng em hoàn thiện hơn về kiến thức, tư duy nghiên cứu và kỹ năng thực hành.

Bên cạnh đó, nhóm cũng xin chân thành cảm ơn các thầy cô Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh đã truyền đạt cho chúng em nền tảng kiến thức vững chắc trong suốt quá trình học tập, tạo điều kiện thuận lợi để chúng em có thể vận dụng vào việc thực hiện đề tài.

Nhóm cũng xin gửi lời cảm ơn đến gia đình và bạn bè, những người luôn ở bên động viên, khích lệ và tiếp thêm động lực để chúng em vượt qua những khó khăn trong quá trình học tập và nghiên cứu. Sự tin tưởng và ủng hộ của mọi người chính là nguồn động lực to lớn giúp chúng em hoàn thành giai đoạn đề án chuyên ngành.

Cuối cùng, nhóm xin gửi lời cảm ơn chân thành đến tất cả những ai đã đóng góp, hỗ trợ dù trực tiếp hay gián tiếp cho sự thành công của đề án này.

Nhóm thực hiện,

Phạm Quốc Toàn – Nguyễn Ngọc Quế Chi – Trần Nguyễn Anh Khoa

Lời cam đoan

Nhóm chúng em xin cam đoan rằng đề tài “Hệ thống Thương mại Điện tử cho Bán lẻ Điện thoại tích hợp Trợ lý Ảo AI Agent” là kết quả tìm hiểu, nghiên cứu và thực hiện của chính nhóm chúng em, dưới sự hướng dẫn của Phó Giáo sư, Tiến sĩ Quản Thành Thơ, thuộc Khoa Khoa học và Kỹ thuật Máy tính – Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh.

Toàn bộ nội dung, số liệu, hình ảnh và kết quả trình bày trong đồ án đều do nhóm tự thực hiện, tổng hợp từ quá trình nghiên cứu, thiết kế và triển khai hệ thống. Những tài liệu, công trình, hay kết quả nghiên cứu của người khác được sử dụng hoặc tham khảo trong đồ án đều đã được trích dẫn và ghi nguồn đầy đủ, đúng quy định.

Nhóm hoàn toàn chịu trách nhiệm về tính trung thực và tính hợp pháp của toàn bộ nội dung trong đồ án này. Nếu có bất kỳ sai phạm nào liên quan đến đạo văn, vi phạm bản quyền hoặc gian lận học thuật, nhóm xin hoàn toàn chịu mọi hình thức xử lý theo quy định của Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh.

TP. Hồ Chí Minh, Tháng 12/2025

Nhóm thực hiện,

Phạm Quốc Toàn – Nguyễn Ngọc Quế Chi – Trần Nguyễn Anh Khoa

Tóm tắt

Đề tài “Hệ thống Thương mại Điện tử cho Bán lẻ Điện thoại tích hợp Trợ lý Ảo AI Agent” hướng đến việc phát triển một nền tảng mua sắm trực tuyến thông minh, ứng dụng công nghệ trí tuệ nhân tạo nhằm nâng cao trải nghiệm người dùng trong lĩnh vực bán lẻ điện thoại và các đồ dùng công nghệ. Hệ thống được định hướng hoạt động như một nền tảng thương mại điện tử toàn diện, nơi người dùng có thể tìm kiếm, tra cứu, đặt hàng và theo dõi tình trạng đơn hàng thông qua các thao tác tương tác trực tiếp với Trợ lý Ảo AI Agent.

Trong giai đoạn đầu, nhóm tập trung vào quá trình khảo sát nhu cầu thực tế, phân tích yêu cầu của người dùng và doanh nghiệp, từ đó xây dựng kiến trúc tổng thể của hệ thống bao gồm các thành phần chức năng chính, mô hình dữ liệu và luồng xử lý nghiệp vụ. Trợ lý Ảo AI Agent được thiết kế dựa trên công nghệ xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP), cho phép người dùng giao tiếp với hệ thống bằng ngôn ngữ tự nhiên để nhận tư vấn sản phẩm, hỗ trợ đặt hàng hoặc kiểm tra thông tin cá nhân.

Trong phạm vi báo cáo này, nhóm đã hoàn thiện bộ mô hình thiết kế hệ thống bao gồm: phân tích yêu cầu, thiết kế kiến trúc, mô hình cơ sở dữ liệu (ERD, schema) và giao diện mô phỏng trải nghiệm người dùng (mockup, wireframe).

Kết quả đạt được của giai đoạn này là bản thiết kế hệ thống hoàn chỉnh, đóng vai trò nền tảng cho giai đoạn tiếp theo – khi nhóm sẽ tiến hành hiện thực hóa hệ thống, tích hợp Trợ lý Ảo AI Agent thực tế, kiểm thử toàn diện và đánh giá mức độ hiệu quả, khả năng mở rộng cũng như mức độ hài lòng của người dùng.

Mục lục

1	Giới thiệu đề tài	1
1.1	Lý do chọn đề tài	1
1.2	Mục tiêu đề tài	1
1.3	Phạm vi đề tài	2
2	Khảo sát thị trường	3
2.1	CellphoneS	3
2.2	FPT Shop	4
2.3	Thế Giới Di Động	6
2.4	So sánh tính năng giữa các nền tảng hiện có và hệ thống đề xuất	7
3	Phân tích yêu cầu	9
4	Cơ sở lý thuyết	10
4.1	Lý thuyết về kiến trúc hệ thống	10
4.1.1	Kiến trúc Monolithic	10
4.1.2	Kiến trúc Microservices	10
4.1.3	Kiến trúc EDA + Cloud-Native	11
4.1.4	So sánh giữa các kiến trúc hệ thống	12
4.2	Lý thuyết về AI Agent	13
4.2.1	Trí tuệ nhân tạo	13
4.2.2	AI Agent	13
4.2.3	RAG	13
4.2.4	Đồ thị tri thức	14
4.2.5	Hệ thống đề xuất	14
5	Công nghệ	15
5.1	AI Agent	15
5.1.1	LangGraph	15

5.1.2	Milvus	15
5.1.3	Neo4j	16
6	Thiết kế hệ thống	17
6.1	Thiết kế kiến trúc	17
6.1.1	Lựa chọn kiến trúc	17
6.1.2	Các lựa chọn thiết kế trong kiến trúc EDA + Cloud-Native . . .	17
6.1.3	Các tầng của hệ thống	22
7	Kết luận và hướng phát triển	25
8	Phụ lục	27

Danh sách hình vẽ

1	Trang chủ của CellphoneS	3
2	Trang chủ của FPT Shop	4
3	Trợ lý ảo của FPT Shop	5
4	Trang chủ của Thế Giới Di Động	6
5	Trợ lý ảo của Thế Giới Di Động	7
6	Sơ đồ kiến trúc hệ thống	22

Danh sách bảng

1	So sánh tính năng giữa các nền tảng hiện có và hệ thống đề xuất	7
2	So sánh giữa các kiến trúc hệ thống	12

1 Giới thiệu đề tài

Trong chương này, nhóm trình bày tổng quan về đề tài nghiên cứu, bao gồm bối cảnh và lý do lựa chọn đề tài, mục tiêu cần đạt được cũng như phạm vi thực hiện của hệ thống. Thông qua đó, chương này nhằm xác định rõ ý nghĩa, hướng phát triển và giới hạn của đề tài.

1.1 Lý do chọn đề tài

Sự phát triển mạnh mẽ của thương mại điện tử trong những năm gần đây đã làm thay đổi toàn diện cách thức mua sắm và tiêu dùng của người dùng. Đặc biệt trong lĩnh vực bán lẻ điện thoại và thiết bị công nghệ, các doanh nghiệp lớn trong nước như CellphoneS, FPT Shop, Thế Giới Di Động đã xây dựng những nền tảng thương mại điện tử hiện đại, đáp ứng đa dạng nhu cầu từ việc tra cứu sản phẩm, so sánh giá, đến đặt hàng trực tuyến. Các hệ thống này được đầu tư bài bản cả về giao diện, trải nghiệm người dùng (UI/UX) lẫn hạ tầng kỹ thuật, mang lại sự tiện lợi và tin cậy cho khách hàng.

Tuy nhiên, khi khảo sát và trải nghiệm thực tế, nhóm nhận thấy rằng các nền tảng thương mại điện tử hiện nay vẫn chủ yếu hoạt động theo cơ chế tương tác truyền thống, nơi người dùng cần thao tác thủ công để tìm kiếm sản phẩm hoặc truy cập thông tin. Một số trang web đã tích hợp chatbot hoặc trợ lý ảo cơ bản, tuy nhiên các hệ thống này chỉ dừng lại ở mức trả lời câu hỏi hoặc hỗ trợ tư vấn đơn giản, chưa có khả năng thực hiện các tác vụ có mức độ tương tác sâu như đặt hàng, hủy đơn hàng, kiểm tra giỏ hàng, hoặc cung cấp thông tin mà khách hàng quan tâm một cách nhanh chóng và an toàn.

Trong bối cảnh trí tuệ nhân tạo (AI) và xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP) ngày càng phát triển, việc tích hợp Trợ lý Ảo thông minh vào hệ thống thương mại điện tử mở ra hướng tiếp cận mới – nơi người dùng có thể tương tác trực tiếp với hệ thống bằng ngôn ngữ tự nhiên, thay vì qua các bước điều hướng phức tạp. Điều này không chỉ giúp tăng tính cá nhân hóa trong trải nghiệm mua sắm, mà còn giảm tải cho nhân viên chăm sóc khách hàng, đồng thời nâng cao hiệu quả kinh doanh cho doanh nghiệp.

Xuất phát từ thực tế đó, nhóm quyết định lựa chọn và thực hiện đề tài “Hệ thống Thương mại Điện tử cho Bán lẻ Điện thoại tích hợp Trợ lý Ảo AI Agent” với mục tiêu xây dựng một mô hình thương mại điện tử thông minh, có khả năng tương tác tự nhiên, hiểu ngữ cảnh, hỗ trợ và thực hiện hành động thay cho người dùng. Đề tài không chỉ mang ý nghĩa thực tiễn trong việc ứng dụng công nghệ AI vào thương mại điện tử, mà còn góp phần nghiên cứu và phát triển các mô hình tương tác người – máy thế hệ mới.

1.2 Mục tiêu đề tài

Mục tiêu của đề tài là thiết kế và phát triển một hệ thống thương mại điện tử hoàn chỉnh, có khả năng hoạt động ổn định, dễ sử dụng, và tích hợp Trợ lý Ảo AI Agent để hỗ trợ người dùng trong quá trình mua sắm trực tuyến. Cụ thể, các mục tiêu chính của đề tài

bao gồm:

- Xây dựng một hệ thống thương mại điện tử chuyên biệt cho lĩnh vực bán lẻ điện thoại và thiết bị công nghệ. Hệ thống cần đáp ứng đầy đủ các chức năng cơ bản như hiển thị sản phẩm, tìm kiếm, quản lý giỏ hàng, đặt hàng, hủy đơn hàng, thanh toán và theo dõi trạng thái đơn hàng. Giao diện người dùng được thiết kế trực quan và thân thiện.
- Phát triển Trợ lý Ảo AI Agent có khả năng giao tiếp tự nhiên với người dùng bằng tiếng Việt, tư vấn sản phẩm mà khách hàng quan tâm và thực hiện các tác vụ mà khách hàng yêu cầu.
- Tích hợp Trợ lý Ảo vào hệ thống web thương mại điện tử nhằm tạo ra trải nghiệm liền mạch cho người dùng. Trợ lý Ảo có thể thực hiện các hành động trực tiếp trên hệ thống như đặt hàng, kiểm tra giỏ hàng hoặc hủy đơn theo yêu cầu, giúp tối ưu hóa quy trình mua sắm.
- Phát triển giao diện và chức năng quản trị cho phép quản lý hệ thống hiệu quả. Quản trị viên có thể thêm, chỉnh sửa, hoặc xóa sản phẩm; theo dõi đơn hàng; quản lý người dùng; và giám sát hoạt động của Trợ lý Ảo. Điều này đảm bảo hệ thống hoạt động ổn định, dễ bảo trì và dễ mở rộng trong tương lai.

1.3 Phạm vi đề tài

Phạm vi thực hiện của đề tài được xác định như sau:

- **Website thương mại điện tử:**

Hệ thống được thiết kế dưới dạng ứng dụng web, hướng đến người dùng truy cập qua trình duyệt trên máy tính. Các chức năng chính của website bao gồm:

- Hiển thị danh sách và chi tiết sản phẩm.
- Quản lý giỏ hàng, đặt hàng, hủy đơn và theo dõi trạng thái đơn hàng.
- Quản lý tài khoản người dùng (đăng ký, đăng nhập, cập nhật thông tin).
- Trang quản trị dành cho admin để quản lý sản phẩm, người dùng và đơn hàng.

- **Trợ lý Ảo AI Agent:**

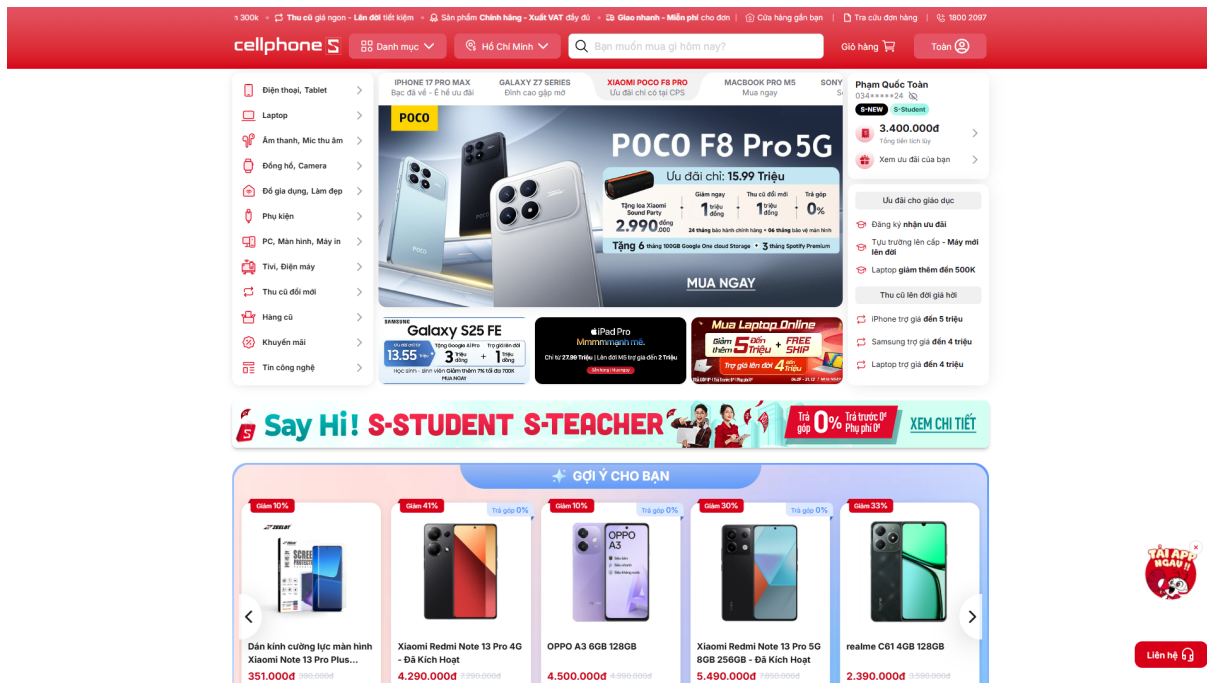
Trợ lý Ảo được phát triển nhằm hỗ trợ người dùng trong quá trình tương tác với hệ thống, đóng vai trò như một nhân viên bán hàng tự động. Các khả năng trong phạm vi đề tài bao gồm:

- Hiểu và phản hồi các truy vấn tiếng Việt bằng ngôn ngữ tự nhiên.
- Hỗ trợ tìm kiếm sản phẩm, gợi ý theo nhu cầu người dùng.
- Thực hiện các hành động có tính tương tác sâu như đặt hàng, kiểm tra hoặc hủy đơn hàng.
- Cung cấp thông tin về chính sách, chương trình khuyến mãi hoặc hướng dẫn sử dụng sản phẩm.

2 Khảo sát thị trường

Trong chương này, nhóm khảo sát, tìm hiểu và trải nghiệm các hệ thống tương tự hiện có trên thị trường. Sau đó, nhóm so sánh các hệ thống và tìm ra khoảng trống trên thị trường. Thông qua đó, xác định hướng phát triển hợp lý của đề tài.

2.1 CellphoneS



Hình 1: Trang chủ của CellphoneS

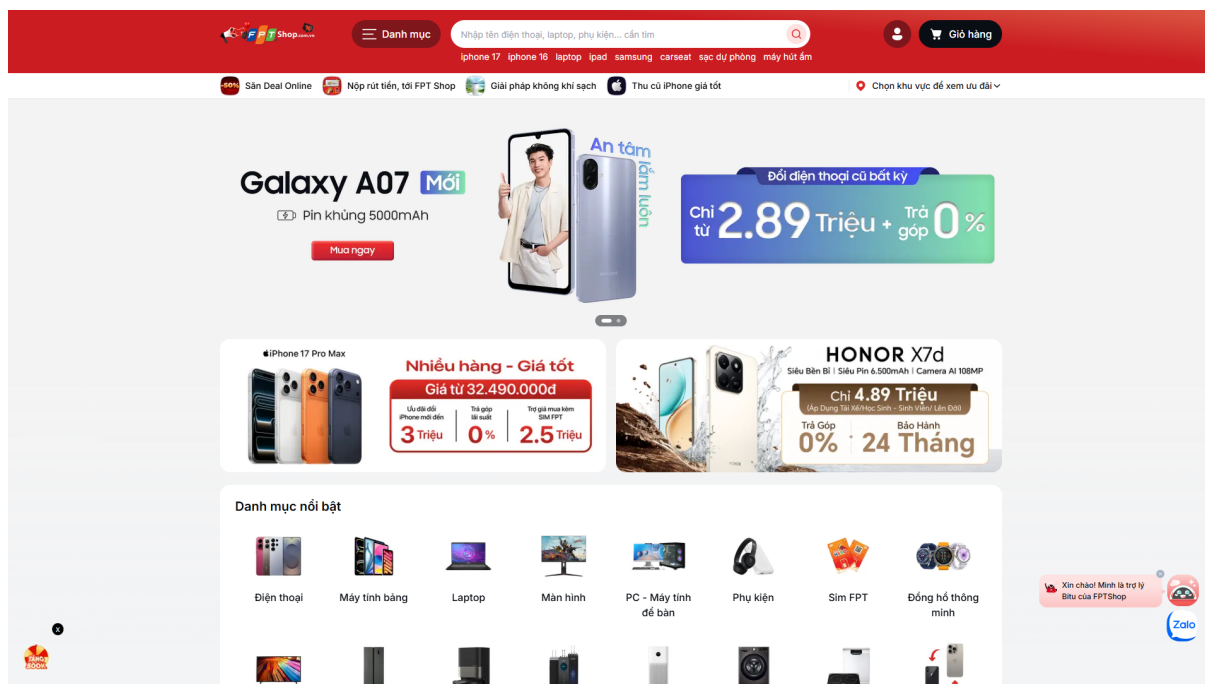
CellphoneS là hệ thống bán lẻ điện thoại, laptop, máy tính bảng và nhiều phụ kiện công nghệ phổ biến tại Việt Nam, hoạt động song song giữa chuỗi cửa hàng và nền tảng thương mại điện tử. Website của CellphoneS hướng tới trải nghiệm mua sắm nhanh, tiện lợi, hỗ trợ trả góp, chương trình thu cũ đổi mới (trade-in) và hệ thống tích điểm khách hàng thân thiết Smember.

Tính năng nổi bật:

- Bộ lọc và tìm kiếm sản phẩm theo giá, thương hiệu, thông số kỹ thuật và chương trình khuyến mãi.
- Hỗ trợ trade-in với quy trình định giá trực tuyến và phản hồi nhanh.
- Trả góp linh hoạt qua nhiều hình thức như thẻ tín dụng, công ty tài chính hoặc ví điện tử.
- Ứng dụng Smember giúp người dùng quản lý điểm thưởng, theo dõi ưu đãi và thông báo khuyến mãi.

Website của CellphoneS thể hiện thể mạnh rõ rệt ở việc xây dựng quy trình trade-in minh bạch và dễ sử dụng, cùng với hệ thống trả góp linh hoạt, giúp người dùng có nhiều lựa chọn tài chính phù hợp. Nền tảng Smember tạo ra hệ sinh thái khách hàng thân thiết, góp phần giữ chân người dùng lâu dài. Tuy nhiên, website vẫn còn hạn chế ở khả năng cá nhân hóa trải nghiệm mua sắm. Các tính năng như đề xuất sản phẩm thông minh hay chatbot hỗ trợ tương tác tự nhiên chưa được đầu tư sâu, khiến hệ thống vẫn chủ yếu hoạt động theo cơ chế tìm kiếm và thao tác thủ công.

2.2 FPT Shop



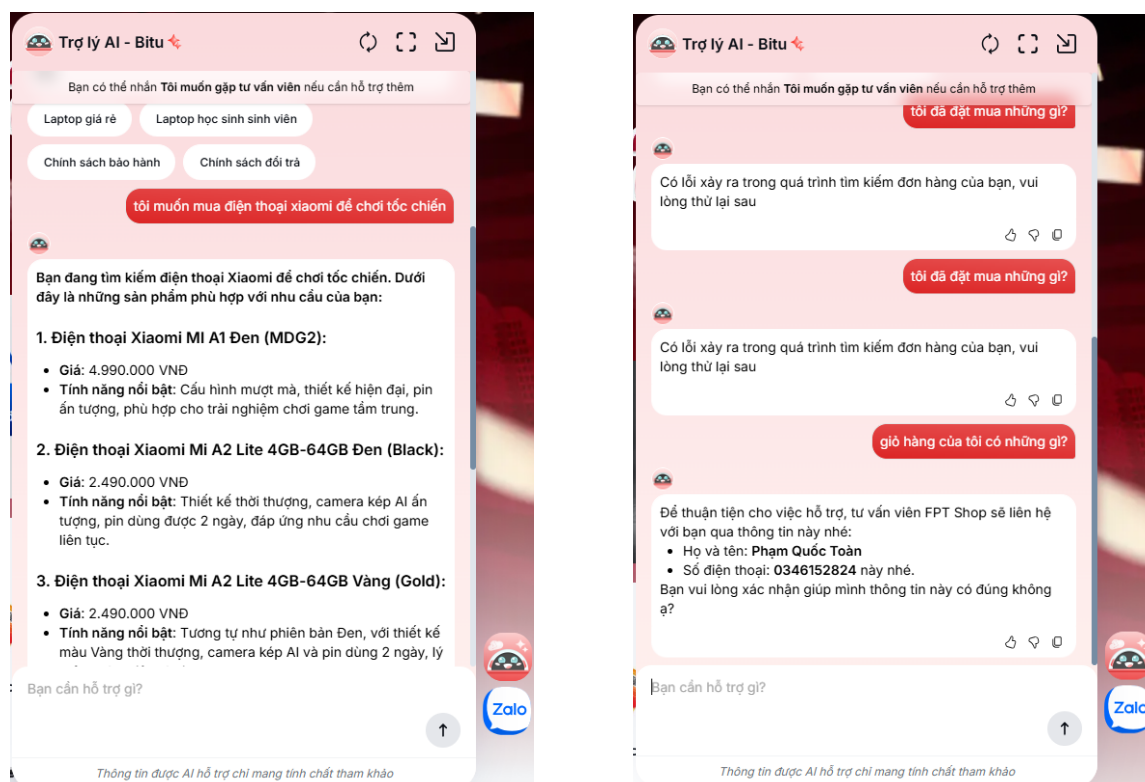
Hình 2: Trang chủ của FPT Shop

FPT Shop là thương hiệu bán lẻ thiết bị công nghệ thuộc Tập đoàn FPT, với mạng lưới cửa hàng rộng khắp và nền tảng thương mại điện tử tích hợp mạnh mẽ. Website của FPT Shop chú trọng trải nghiệm người dùng, hỗ trợ trả góp, khuyến mãi và dịch vụ hậu mãi, đồng thời ứng dụng công nghệ AI trong tư vấn, hỗ trợ khách hàng.

Tính năng nổi bật:

- Hệ thống tìm kiếm và lọc sản phẩm mạnh, hiển thị nhiều biến thể sản phẩm và cho phép so sánh cơ bản.
- Hỗ trợ trả góp đa dạng với thông tin minh bạch về lãi suất và các tùy chọn thanh toán.
- Ứng dụng di động cung cấp thông tin khuyến mãi, voucher và tích điểm khách hàng thân thiết.
- Tích hợp chatbot AI hỗ trợ khách hàng 24/7.

FPT Shop sở hữu nền tảng công nghệ hiện đại với khả năng đồng bộ tốt giữa kênh bán hàng trực tuyến và hệ thống cửa hàng vật lý. Trợ lý ảo có khả năng tư vấn sản phẩm, giúp cải thiện tốc độ phản hồi và giảm tải cho nhân viên tư vấn. Tuy nhiên, trợ lý ảo không thể hỗ trợ khách hàng trong việc thực hiện các tác vụ có tính tương tác sâu như: Đặt hàng, hủy đơn, ... Do đó, hệ thống hỗ trợ khách hàng chủ yếu được triển khai thông qua việc trò chuyện với nhân viên tư vấn trực tuyến cho các tác vụ tương tác sâu thay vì chatbot tự động. Cách tiếp cận này giúp tăng tính chính xác trong phản hồi, song không đảm bảo khả năng phản hồi nhanh và tự động hóa quy trình chăm sóc khách hàng khi lưu lượng truy cập tăng cao.



Hình 3: Trợ lý ảo của FPT Shop

2.3 Thế Giới Di Động



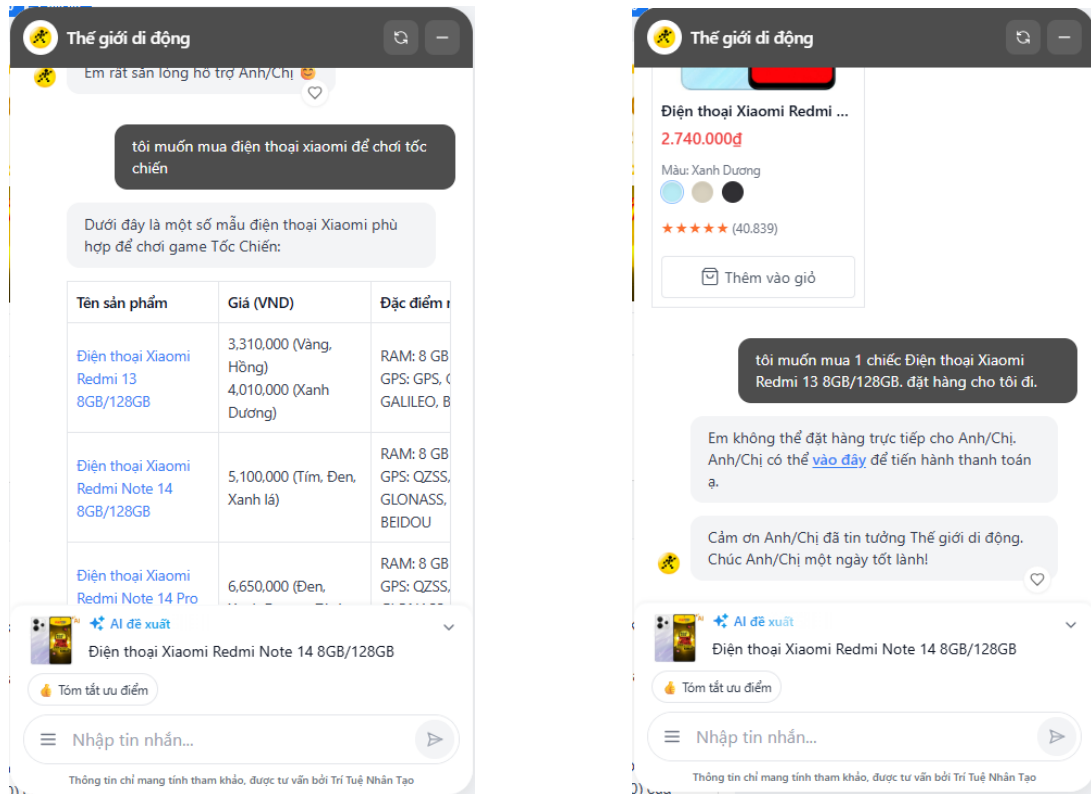
Hình 4: Trang chủ của Thế Giới Di Động

Thế Giới Di Động là hệ thống bán lẻ điện thoại và thiết bị điện tử tiêu dùng lớn nhất Việt Nam, với hàng nghìn cửa hàng trên toàn quốc. Website được tối ưu cho người dùng phổ thông, tập trung vào quy trình mua nhanh, khuyến mãi hấp dẫn và chính sách hậu mãi rõ ràng.

Tính năng nổi bật:

- Bộ lọc sản phẩm đa tiêu chí, gợi ý kết quả tức thì và hiển thị sản phẩm nổi bật.
- Chính sách giao hàng, đổi trả, bảo hành minh bạch.
- Hệ thống khuyến mãi, flash sale và combo sản phẩm trình bày nổi bật.
- Hỗ trợ khách hàng đa kênh: hotline, chat trực tuyến, mạng xã hội và cửa hàng.

Website của Thế Giới Di Động có thiết kế thân thiện, tốc độ truy cập cao và khuyến mãi cập nhật liên tục. Bên cạnh đó, Thế Giới Di Động đã tích hợp trợ lý ảo hỗ trợ khách hàng. Tuy nhiên, cũng tương tự như hệ thống của FPT Shop, các trợ lý ảo này chỉ dừng lại ở mức trả lời câu hỏi hoặc hướng dẫn cơ bản, chẳng hạn như cung cấp thông tin khuyến mãi, tra cứu hoặc tư vấn sản phẩm. Các trợ lý ảo này chưa thực hiện được các hành động trực tiếp trên hệ thống như đặt hàng, thêm sản phẩm vào giỏ, hoặc hủy đơn.



Hình 5: Trợ lý ảo của Thế Giới Di Động

2.4 So sánh tính năng giữa các nền tảng hiện có và hệ thống đề xuất

Bảng 1: So sánh tính năng giữa các nền tảng hiện có và hệ thống đề xuất

Nhóm tính năng	CellphonesS	FPT Shop	Thế Giới Di Động	Hệ thống đề xuất
Tìm kiếm và lọc sản phẩm	Có	Có	Có	Có
Phân loại và hiển thị sản phẩm	Có	Có	Có	Có
Đánh giá và phản hồi sản phẩm	Có	Có	Có	Có
Quản lý giỏ hàng và đơn hàng	Có	Có	Có	Có
Hệ thống khuyến mãi và ưu đãi	Có	Có	Có	Có
Giao diện thân thiện, tối ưu UX/UI	Có	Có	Có	Có
Tư vấn sản phẩm	Có	Có	Có	Có
Hỗ trợ thao tác (đặt hàng, hủy đơn, ...)	Không	Không	Không	Có

Từ kết quả khảo sát ba nền tảng thương mại điện tử lớn tại Việt Nam, có thể nhận thấy rằng mỗi hệ thống đều có định hướng riêng trong việc phát triển và phục vụ người dùng.

Tuy nhiên, nhóm nhận thấy rằng mặc dù các doanh nghiệp trong ngành đã có bước tiến lớn trong việc số hóa hoạt động bán lẻ, các nền tảng thương mại điện tử trong mảng bán lẻ điện thoại di động và các đồ dùng công nghệ tại Việt Nam chưa thật sự tích hợp Trợ lý Ảo thông minh có khả năng giao tiếp tự nhiên và thực hiện các hành động có tính tương tác sâu trên hệ thống.

Khoảng trống này mở ra cơ hội để nhóm nghiên cứu và phát triển một mô hình thương mại điện tử tích hợp Trợ lý Ảo AI Agent, hướng đến việc nâng cao trải nghiệm người dùng, tự động hóa quy trình giao dịch, và tăng tính cá nhân hóa trong tương tác mua sắm.



3 Phân tích yêu cầu

Trong chương này, nhóm ...

4 Cơ sở lý thuyết

Chương này tập trung vào các cơ sở lý thuyết quan trọng về kiến trúc hệ thống và AI Agent. Những cơ sở lý thuyết này đóng vai trò làm nền tảng cho việc thực hiện đề tài.

4.1 Lý thuyết về kiến trúc hệ thống

4.1.1. Kiến trúc Monolithic

Kiến trúc Monolithic là mô hình kiến trúc trong đó toàn bộ các phần của hệ thống như giao diện, xử lý nghiệp vụ và truy cập dữ liệu được gộp chung trong một ứng dụng duy nhất. Ứng dụng được xây dựng và triển khai như một khối thống nhất. Mọi thành phần của hệ thống chạy trong cùng tiến trình, dùng chung tài nguyên và phụ thuộc chặt chẽ vào nhau.

Đặc điểm:

- Mọi chức năng nằm trong **một mã nguồn và một tiến trình chạy**[1].
- Các phần hệ thống phụ thuộc chặt chẽ vào nhau.
- Thường tổ chức theo các tầng: Presentation – Business – Data Access[1].

Ưu điểm:

- Dễ phát triển ban đầu, phù hợp nhóm nhỏ.
- Triển khai đơn giản, ít yêu cầu hạ tầng[1].
- Hiệu năng tốt cho hệ thống nhỏ.
- Kiểm thử tổng thể dễ dàng.

Nhược điểm:

- Khó mở rộng, phải mở rộng cả hệ thống.
- Khó bảo trì khi lớn dần, dễ bị rối.
- Triển khai chậm, khi cần thay đổi nhỏ cũng phải re-deploy toàn bộ hệ thống.
- Khả năng chịu lỗi thấp.
- Khó áp dụng công nghệ mới cho từng phần riêng.

4.1.2. Kiến trúc Microservices

Kiến trúc Microservices là kiến trúc mà ở đó hệ thống được phân tách thành các dịch vụ (service) nhỏ, độc lập — mỗi dịch vụ đảm nhận một chức năng riêng và có thể phát triển cũng như triển khai mà không ảnh hưởng đến phần còn lại.

Các dịch vụ giao tiếp với nhau qua API hoặc các kênh trao đổi nhẹ, cho phép hệ thống linh hoạt, dễ mở rộng, dễ bảo trì và thuận tiện khi cần nâng cấp hoặc mở rộng từng phần riêng lẻ.

Đặc điểm:

- Mỗi dịch vụ có **mã nguồn và vòng đời triển khai riêng**[2].
- Dịch vụ có thể dùng **ngôn ngữ hoặc công nghệ khác nhau**.
- Dễ phân tách theo nghiệp vụ, phù hợp tổ chức theo nhóm nhỏ độc lập[2].

Ưu điểm:

- Dễ mở rộng, có thể mở rộng từng dịch vụ theo nhu cầu.
- Triển khai linh hoạt, thay đổi dịch vụ này không ảnh hưởng dịch vụ khác.
- Tăng tính ổn định, lỗi dịch vụ không làm sập toàn hệ thống.
- Cho phép áp dụng công nghệ khác nhau cho từng phần.

Nhược điểm:

- Phức tạp hơn kiến trúc Monolithic, đòi hỏi DevOps và hạ tầng tốt.
- Theo dõi và kiểm thử khó hơn vì hệ thống phân tán[2].
- Chi phí giao tiếp giữa các dịch vụ cao hơn[2].
- Cần thêm các thành phần như API Gateway, dịch vụ quản lý cấu hình, giám sát phân tán.

4.1.3. Kiến trúc EDA + Cloud-Native

Kiến trúc EDA (Event-Driven Architecture) + Cloud-Native được phát triển dựa trên kiến trúc Microservices mà ở đó có hai điểm khác biệt rõ nhất đó là tổ chức hệ thống theo mô hình phát sinh sự kiện (event) – truyền sự kiện – xử lý sự kiện và hướng tới xây dựng ứng dụng tối ưu cho môi trường điện toán đám mây, thường dựa trên container và nền tảng điều phối như Docker hoặc Kubernetes.

Khi kết hợp, hệ thống trở nên linh hoạt, mở rộng tốt, xử lý bất đồng bộ hiệu quả và phù hợp với môi trường phân tán hiện đại.

Đặc điểm:

- Giao tiếp qua sự kiện, thông qua hàng đợi hoặc hệ thống truyền tin[3].
- Sử dụng container và hoạt động trên môi trường Cloud nên có khả năng **tự động mở rộng, tự phục hồi**[3].

- Hoạt động tốt trong môi trường nhiều dịch vụ, khối lượng dữ liệu lớn và thay đổi nhanh.

Ưu điểm:

- Xử lý bất đồng bộ, phù hợp tải cao[3].
- Khả năng mở rộng rất tốt, tự tăng/giảm tài nguyên theo nhu cầu[3].
- Tính chịu lỗi cao, sự kiện được lưu lại ngay cả khi dịch vụ xử lý tạm lỗi[3].
- Dễ thêm chức năng mới, chỉ cần thêm dịch vụ tiêu thụ sự kiện.
- Tối ưu chi phí, phân bổ tài nguyên linh hoạt.

Nhược điểm:

- Thiết kế và vận hành phức tạp, đòi hỏi hiểu biết sâu về truyền tin, hàng đợi, container[3].
- Khó theo dõi luồng xử lý, cần công cụ giám sát mạnh.
- Việc thiết kế sự kiện không hợp lý có thể gây trùng dữ liệu hoặc rối luồng xử lý[3].
- Cần hạ tầng cloud và DevOps tốt[3].

4.1.4. So sánh giữa các kiến trúc hệ thống

Bảng 2: So sánh giữa các kiến trúc hệ thống

Tiêu chí	Monolithic	Microservices	EDA + Cloud-Native
Khả năng mở rộng	Tệ	Tốt	Rất tốt
Mức độ phức tạp	Thấp	Trung bình	Cao
Độ tin cậy	Thấp	Ổn	Rất tốt
Chi phí vận hành	Thấp	Trung bình	Cao
Tốc độ triển khai	Chậm	Nhanh	Rất nhanh
Tính linh hoạt	Thấp	Cao	Rất cao
Phù hợp quy mô hệ thống	Nhỏ – trung bình	Lớn	Rất lớn / phân tán
Hiệu quả khi chạy trên Cloud	Tệ	Tốt	Rất tốt

Nhìn chung, các kiến trúc Monolithic, Microservices và EDA + Cloud-Native khác nhau chủ yếu ở mức độ phân tách thành phần và cách các thành phần giao tiếp với nhau. Mỗi kiến trúc thể hiện một mức độ tiến hóa khác nhau trong cách tổ chức hệ thống phần mềm hiện đại. Trong tiến trình phát triển kiến trúc phần mềm, xu hướng đang chuyển dần từ mô hình nguyên khối sang các mô hình phân tán và sự kiện nhằm đáp ứng nhu cầu mở rộng, độ tin cậy và tính linh hoạt cao hơn. Các kiến trúc hiện đại như Microservices và EDA + Cloud-Native được xem là phù hợp với môi trường triển khai đa nền tảng, tự động hóa và điện toán đám mây.

4.2 Lý thuyết về AI Agent

4.2.1. Trí tuệ nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence) là các hệ thống được nghiên cứu và xây dựng để có khả năng thực hiện những nhiệm vụ vốn đòi hỏi trí tuệ con người, như học tập, lập luận, hiểu ngôn ngữ hoặc đưa ra quyết định. Mục tiêu của AI là mô phỏng quá trình nhận thức của con người thông qua mô hình toán học và thuật toán[4].

Các lĩnh vực chính của AI gồm:

- Học máy (Machine Learning)
- Thị giác máy tính (Computer Vision)
- Xử lý ngôn ngữ tự nhiên (Natural Language Processing)

Phần lớn hệ thống AI hiện nay là AI hẹp (Narrow AI)[4], hoạt động tối ưu cho một nhiệm vụ cụ thể như phân loại dữ liệu, nhận diện giọng nói hoặc tư vấn khách hàng.

4.2.2. AI Agent

AI Agent là mô hình Trí tuệ nhân tạo được mở rộng thêm khả năng hành động thông qua các công cụ được tích hợp. Thay vì chỉ sinh văn bản, Agent có thể phân tích yêu cầu, lập kế hoạch, lựa chọn hành động phù hợp và truy xuất dữ liệu thực tế từ các nguồn bên ngoài[5].

Một AI Agent điển hình bao gồm:

- **Bộ tư duy (reasoning loop)** để phân tích và hiểu mục tiêu[5].
- **Bộ lập kế hoạch (planner)** để xác định các bước thực hiện[5].
- **Cơ chế tool/function calling** để gọi API, truy vấn cơ sở dữ liệu hoặc xử lý thông tin[5].

4.2.3. RAG

RAG (Retrieval-Augmented Generation) là kỹ thuật kết hợp mô hình ngôn ngữ với hệ thống truy xuất thông tin nhằm nâng cao độ chính xác và tính nhất quán của câu trả lời. Thay vì dựa hoàn toàn vào kiến thức có sẵn trong mô hình, RAG tìm kiếm các tài liệu liên quan từ nguồn dữ liệu bên ngoài rồi đưa chúng vào ngữ cảnh (context) để mô hình sinh câu trả lời[6].

Quy trình RAG gồm ba bước:

- **Embedding:** chuyển tài liệu thành dạng vector số[6].
- **Retrieval:** tìm các vector gần nhất với vector ứng với câu hỏi của người dùng[6].

- **Augmentation:** đưa nội dung truy xuất được vào ngữ cảnh trước khi cho mô hình trả lời[6].

RAG đặc biệt hữu ích trong các hệ thống yêu cầu thông tin chính xác, cập nhật và bám sát dữ liệu nội bộ, giúp giảm thiểu hiện tượng mô hình sinh ra những thông tin, sự kiện không có thật[6].

4.2.4. Đồ thị tri thức

Đồ thị tri thức (Knowledge Graph) là mô hình biểu diễn tri thức dưới dạng đồ thị, trong đó các thực thể được biểu diễn bằng các nút (nodes) và các quan hệ giữa chúng bằng cạnh (edges)[7]. Cấu trúc này cho phép lưu trữ thông tin có tính liên kết cao và hỗ trợ suy luận logic.

Một đồ thị tri thức thường bao gồm:

- **Nút:** để biểu diễn các thực thể có dữ liệu và quan hệ với các thực thể khác[7].
- **Cạnh:** để thể hiện mối quan hệ giữa các nút[7].
- **Nhãn (Label):** lưu trữ thông tin, dữ liệu của nút hoặc cạnh[7].

Đồ thị tri thức giúp các hệ thống giảm thời gian truy xuất các dữ liệu có tính liên kết cao. Bên cạnh đó, nhờ đặc tính lưu trữ dữ liệu theo quan hệ nên đồ thị tri thức mang lại khả năng suy luận ra các tri thức, quan hệ mới dựa trên các quan hệ đã có.

4.2.5. Hệ thống đề xuất

Hệ thống đề xuất (Recommendation System) là những hệ thống nhằm cung cấp trải nghiệm cá nhân hóa cho người dùng bằng cách đề xuất các sản phẩm phù hợp với nhu cầu hoặc sở thích. Các phương pháp phổ biến bao gồm:

- **Lọc cộng tác (Collaborative filtering):** lọc và đề xuất gợi ý cho một người dùng dựa trên sự tương đồng về đặc điểm và hành vi người dùng của họ so với những người dùng đã sử dụng sản phẩm, dịch vụ[8].
- **Lọc theo nội dung (Content-based filtering):** lọc và đề xuất gợi ý cho một người dùng dựa trên đặc điểm của các sản phẩm, dịch vụ mà họ đã sử dụng[8].
- **Phương pháp đề xuất lai (Hybrid):** kết hợp hai phương pháp trên nhằm tối ưu độ chính xác[8].

Hệ thống đề xuất có ý nghĩa to lớn cho cả doanh nghiệp và người dùng. Hệ thống đề xuất giúp cải thiện trải nghiệm người dùng, từ đó giúp tăng khả năng giữ chân khách hàng và nâng cao tỉ lệ chuyển đổi từ người dùng thành khách hàng[8].

5 Công nghệ

Trong chương này, nhóm giới thiệu về các công nghệ mà nhóm đã tìm hiểu, nghiên cứu và quyết định lựa chọn để thực hiện đồ án.

5.1 AI Agent

5.1.1. LangGraph

LangGraph là framework chuyên dụng để xây dựng AI Agent có khả năng lập luận theo từng bước (*step-by-step reasoning*), duy trì trạng thái (stateful) và phối hợp nhiều công cụ (tools) theo luồng xử lý dạng đồ thị. LangGraph mở rộng mô hình ngôn ngữ lớn (LLM) bằng cách cho phép quản lý logic xử lý rõ ràng và có thể kiểm soát được.

Vai trò trong hệ thống

- Tạo AI Agent có khả năng phân tích yêu cầu của khách hàng.
- Điều phối quá trình reasoning: nhận câu hỏi → xác định nhu cầu → quyết định công cụ cần dùng → thực thi → tổng hợp kết quả.
- Kết nối với Milvus và Neo4j thông qua các tool tùy chỉnh.
- Đảm bảo hệ thống AI hoạt động ổn định, minh bạch và có thể debug.

Lý do lựa chọn

- Hỗ trợ thiết kế workflow linh hoạt và dễ mở rộng.
- Tích hợp tốt với các mô hình LLM hiện đại.
- Tích hợp tốt với nhiều công cụ và cơ sở dữ liệu.
- Tối ưu cho bài toán AI Agent phức tạp thay vì chỉ chat đơn thuần.

5.1.2. Milvus

Milvus là cơ sở dữ liệu vector hiệu năng cao, được thiết kế để lưu trữ và truy vấn các embedding từ dữ liệu phi cấu trúc. Milvus hỗ trợ các thuật toán indexing như IVF, HNSW giúp tìm kiếm tương đồng nhanh chóng và chính xác.

Vai trò trong hệ thống

- Lưu trữ vector của tài liệu phi cấu trúc như mô tả sản phẩm, FAQ, chính sách, hướng dẫn sử dụng.
- Hỗ trợ AI Agent thực hiện RAG để lấy đúng thông tin liên quan từ kho dữ liệu.
- Giảm hiện tượng hallucination của LLM bằng việc cung cấp dữ liệu tham chiếu tin cậy.

Lý do lựa chọn

- Tốc độ tìm kiếm nhanh, ổn định ngay cả khi dữ liệu lớn.
- Hỗ trợ mở rộng theo chiều ngang dễ dàng.
- Tích hợp trực tiếp với hệ sinh thái AI như LangChain, LangGraph, HuggingFace.

5.1.3. Neo4j

Neo4j là hệ quản trị cơ sở dữ liệu đồ thị hàng đầu, được tối ưu cho việc lưu trữ và truy vấn dữ liệu có nhiều mối quan hệ phức tạp. Dữ liệu được tổ chức thành node (thực thể) và relationship (quan hệ), và được truy vấn bằng ngôn ngữ Cypher.

Vai trò trong hệ thống

- Hỗ trợ AI Reasoning nhờ khả năng truy vấn theo quan hệ.
- Hỗ trợ Recommendation System bằng cách tìm sản phẩm liên quan, tương đồng hoặc bổ trợ.
- Cung cấp nền tảng tri thức có cấu trúc giúp AI đưa ra kết luận chính xác hơn.

Lý do lựa chọn

- Hiệu quả cao trong các bài toán quan hệ mà SQL khó tối ưu.
- Dễ mô hình hóa tri thức, trực quan, mở rộng linh hoạt.
- Tương thích tốt với AI Agent thông qua truy vấn Cypher.

6 Thiết kế hệ thống

Trong chương này, nhóm ...

6.1 Thiết kế kiến trúc

6.1.1. Lựa chọn kiến trúc

Hệ thống thương mại điện tử tích hợp Trợ lý Ảo AI của nhóm yêu cầu một kiến trúc **hiện đại, linh hoạt và có khả năng mở rộng**, đủ để xử lý lượng lớn yêu cầu đồng thời, hỗ trợ phản hồi theo thời gian thực (real-time) và dễ dàng mở rộng theo từng phần.

Từ những phân tích về nhu cầu kỹ thuật và khả năng phát triển lâu dài, kiến trúc hệ thống cần đáp ứng các mục tiêu sau:

- Mở rộng linh hoạt cho từng dịch vụ độc lập.
- Hỗ trợ xử lý bất đồng bộ và realtime.
- Đảm bảo tính cô lập giữa các thành phần.
- Cho phép triển khai, cập nhật, bảo trì riêng từng dịch vụ mà không ảnh hưởng toàn hệ thống.
- Duy trì độ tin cậy cao, tránh tình trạng “sập dây chuyền” khi một dịch vụ gặp lỗi.
- Tận dụng hạ tầng Cloud để tự động hóa triển khai, cân bằng tải và tối ưu chi phí.
- Hỗ trợ tích hợp Trợ lý Ảo một cách dễ dàng.

Sau khi tìm hiểu, nghiên cứu về cơ sở lý thuyết của các mô hình kiến trúc phổ biến như *Monolithic*, *Microservices* và *EDA + Cloud-Native*, nhóm lựa chọn kiến trúc **EDA + Cloud-Native** vì những lý do sau:

- Phù hợp với hệ thống có nhiều sự kiện phát sinh và cần xử lý bất đồng bộ như hệ thống thương mại điện tử.
- Đảm bảo khả năng phản hồi nhanh và độ trễ thấp.
- Dễ dàng mở rộng hoặc thêm mới dịch vụ mà không ảnh hưởng hệ thống hiện hữu.
- Giúp giảm phụ thuộc giữa các thành phần và tăng tính mở rộng.
- Cho phép tích hợp AI Agent hoạt động với các nền tảng vector database và graph database một cách dễ dàng.

6.1.2. Các lựa chọn thiết kế trong kiến trúc EDA + Cloud-Native

Các mô hình thiết kế (pattern) trong kiến trúc EDA + Cloud-Native là những mô hình được thiết kế để giúp các dịch vụ giao tiếp và phản ứng với nhau thông qua các sự kiện một cách bất đồng bộ, hiệu quả và an toàn. Cách tiếp cận này giúp xây dựng hệ thống

có khả năng mở rộng cao, độ chịu lỗi tốt và mức độ phụ thuộc thấp giữa các thành phần. Những mô hình này mô tả cách sự kiện được tạo ra, truyền đi và tiêu thụ trong môi trường phân tán, qua đó đáp ứng các yêu cầu về khả năng mở rộng theo tải, độ tin cậy, và tính phản hồi của hệ thống.

Các mô hình tiêu biểu gồm Publish–Subscribe, Message Queue, Event Sourcing, CQRS, Saga và Choreography[3], mỗi mô hình giải quyết một nhóm vấn đề đặc thù trong quá trình thiết kế và vận hành hệ thống theo kiến trúc EDA + Cloud-Native.

Về mặt luân chuyển sự kiện

Các mô hình luân chuyển sự kiện (Event-Delivery Patterns) mô tả cách sự kiện được truyền từ dịch vụ phát sinh (producer) đến dịch vụ xử lý (consumer). Hai mô hình phổ biến nhất là Publish–Subscribe (Pub/Sub) và Message Queue (MQ). Các cơ chế này giúp hệ thống duy trì sự tách biệt giữa các dịch vụ, đảm bảo độ tin cậy và khả năng mở rộng khi vận hành trên hạ tầng phân tán.

Publish – Subscribe (Pub/Sub) là mô hình truyền sự kiện trong đó producer gửi sự kiện lên một kênh (topic), và nhiều consumer có thể đồng thời nhận được sự kiện này. Mô hình này giúp hệ thống linh hoạt hơn và dễ mở rộng. Mô hình này có những đặc điểm sau:

- Producer không cần biết số lượng hay danh tính consumer[3].
- Một sự kiện có thể kích hoạt nhiều dịch vụ cùng lúc[3].
- Mô hình topic hỗ trợ mở rộng theo chiều ngang.
- Thích hợp với xử lý song song nhiều luồng.

Pub/Sub được ứng dụng trong các hệ thống cần phân phối sự kiện đến nhiều dịch vụ cùng lúc, đặc biệt khi nhiều thành phần trong kiến trúc phải phản ứng đồng thời với các thay đổi. Mô hình này hỗ trợ mở rộng linh hoạt, giảm phụ thuộc giữa các dịch vụ và cho phép xử lý song song ở quy mô lớn, nhờ đó phù hợp với những luồng nghiệp vụ đòi hỏi khả năng phản ứng tức thời và lan tỏa.

Message Queue là mô hình hàng đợi giúp đảm bảo mỗi thông điệp chỉ được xử lý bởi một consumer, phù hợp với mô hình truyền sự kiện điểm–đến–điểm (point-to-point). Đặc điểm của mô hình này là:

- Mỗi message được một consumer duy nhất xử lý[3].
- Không mất message nhờ cơ chế lưu trữ tạm.
- Hỗ trợ retry khi lỗi xảy ra.
- Giúp phân phối tải giữa nhiều worker.

Message Queue được sử dụng trong các nghiệp vụ yêu cầu xử lý tuần tự, đảm bảo tính tin cậy và mức độ cam kết cao trong việc tiêu thụ sự kiện. Mô hình này giúp duy trì sự

ổn định của hệ thống bằng cách kiểm soát tải, đảm bảo mỗi sự kiện được xử lý trọn vẹn, đồng thời hạn chế rủi ro mất mát dữ liệu khi dịch vụ gặp sự cố hoặc gián đoạn.

Sau khi phân tích hai mô hình truyền sự kiện phổ biến là Publish/Subscribe và Message Queue, nhóm quyết định lựa chọn Pub/Sub làm mô hình luân chuyển sự kiện cho hệ thống. Việc lựa chọn này xuất phát từ đặc thù của hệ thống thương mại điện tử, nơi mỗi sự kiện thường cần được nhiều dịch vụ cùng lúc lắng nghe và xử lý. Ví dụ, khi một đơn hàng được tạo ra (order_created), Inventory Service cần cập nhật tồn kho, Payment Service tiếp nhận để xử lý thanh toán, Notification Service gửi thông báo cho người dùng, và Analytics Service ghi nhận dữ liệu phục vụ thống kê. Với Pub/Sub, dịch vụ phát sinh sự kiện không cần biết có bao nhiêu dịch vụ quan tâm, và mỗi consumer có thể xử lý độc lập mà không ảnh hưởng đến phần còn lại của hệ thống.

Mô hình Pub/Sub giúp giảm mạnh sự phụ thuộc giữa các dịch vụ và tạo điều kiện cho việc mở rộng hệ thống về sau. Khi bổ sung dịch vụ mới, chỉ cần đăng ký (subscribe) vào topic sự kiện mà không cần thay đổi bất kỳ thành phần đang hoạt động. Điều này phù hợp với tư duy mở rộng linh hoạt của kiến trúc Cloud-Native và giúp hệ thống vận hành bất đồng bộ hiệu quả. Vì các lợi ích trên, nhóm thống nhất lựa chọn Pub/Sub làm mô hình truyền sự kiện duy nhất cho toàn bộ hệ thống.

Về mặt quản lý trạng thái dữ liệu

Trong môi trường phân tán, quản lý trạng thái dữ liệu đóng vai trò then chốt. Do đó, để hạn chế sự bất đồng bộ trong trạng thái dữ liệu giữa các dịch vụ, chúng ta cần thiết kế hệ thống theo các mô hình quản lý trạng thái (State Management Patterns) hiện đại và phù hợp. Hai mô hình thông dụng là Event Sourcing và CQRS, mỗi mô hình phù hợp cho các loại nghiệp vụ khác nhau.

Event Sourcing lưu toàn bộ thay đổi dưới dạng chuỗi sự kiện thay vì trạng thái cuối cùng. Đặc điểm của mô hình này là:

- Lưu lịch sử dữ liệu đầy đủ[3].
- Khôi phục trạng thái bằng replay sự kiện.
- Hỗ trợ kiểm toán và truy vết mạnh[3].
- Yêu cầu thiết kế và lưu trữ phức tạp hơn[3].

Event Sourcing thích hợp cho các hệ thống cần duy trì lịch sử thay đổi chi tiết, hỗ trợ kiểm toán hoặc phục hồi trạng thái với độ chính xác cao. Nhờ lưu lại toàn bộ chuỗi sự kiện, mô hình này cho phép tái xây dựng dữ liệu, phân tích hành vi theo thời gian và đáp ứng các yêu cầu về minh bạch, nhất quán và khả năng truy vết trong môi trường phân tán.

CQRS (Command Query Responsibility Segregation) tách biệt luồng ghi và luồng đọc, giúp tối ưu từng thành phần riêng biệt. Mô hình này có những đặc điểm sau:

- Tách biệt đường đi của đọc và ghi[3].

- Cho phép tối ưu từng luồng theo nhu cầu[3].
- Hỗ trợ mở rộng độc lập.
- Tăng cường bảo mật nhờ phân tách chức năng.

CQRS phù hợp với những hệ thống có tải truy vấn lớn, nơi nhu cầu đọc dữ liệu vượt trội so với ghi. Việc tách biệt hai luồng xử lý giúp tối ưu hiệu năng, cải thiện tốc độ phản hồi và cho phép mở rộng độc lập từng phần của hệ thống, đồng thời tăng cường khả năng kiểm soát truy cập và tính linh hoạt trong thiết kế dữ liệu.

Trong hai mô hình quản lý trạng thái là Event Sourcing và CQRS, nhóm lựa chọn CQRS làm mô hình quản lý trạng thái dữ liệu cho hệ thống. Lý do là hệ thống thương mại điện tử thường có tỷ lệ truy vấn đọc lớn hơn rất nhiều so với tỷ lệ ghi. Người dùng liên tục xem sản phẩm, xem giỏ hàng, xem lịch sử đơn hàng và tra cứu thông tin chi tiết. Các thao tác này yêu cầu phản hồi nhanh, ổn định và có khả năng xử lý tải cao. CQRS cho phép tách phần ghi (command) và phần đọc (query) thành hai thành phần độc lập, giúp tối ưu từng luồng theo đúng đặc thù của nó.

Với CQRS, phần ghi tập trung đảm bảo tính toàn vẹn dữ liệu và xử lý đúng logic nghiệp vụ, trong khi phần đọc có thể được tối ưu bằng cache, denormalized view hoặc cơ sở dữ liệu chuyên biệt cho truy vấn nhanh. Điều này làm giảm tải cho hệ thống ghi và cải thiện tốc độ truy vấn lên giao diện người dùng

Event Sourcing mặc dù mạnh về khả năng lưu vết, nhưng lại phức tạp trong thiết kế và không phù hợp với đề tài của nhóm. Do đó, nhóm chọn CQRS làm mô hình quản lý trạng thái duy nhất để đảm bảo cân bằng giữa hiệu năng, sự đơn giản và khả năng triển khai thực tế.

Về mặt điều phối hệ thống

Các mô hình điều phối (Orchestration Patterns) mô tả cách các dịch vụ phối hợp để xử lý quy trình nghiệp vụ phức tạp. Hai mô hình phổ biến là Saga và Choreography.

Saga chia tác vụ lớn thành nhiều bước nhỏ, mỗi bước có thể rollback khi lỗi xảy ra. Mô hình này có những đặc điểm sau:

- Chia tác vụ thành nhiều bước tuần tự.
- Có hành động để xử lý lỗi nếu có ở từng bước[3].
- Có trung tâm điều phối (orchestrator) để tổ chức thực hiện tác vụ[3].
- Hỗ trợ xử lý lỗi và rollback có kiểm soát.

Saga được áp dụng trong các quy trình nghiệp vụ phức tạp đòi hỏi nhiều bước xử lý có liên quan và cần duy trì tính nhất quán trong môi trường phân tán. Mô hình này hỗ trợ điều phối luồng công việc theo từng giai đoạn, đảm bảo hệ thống có thể xử lý lỗi an toàn, khôi phục từng phần khi cần thiết và duy trì tính toàn vẹn của các giao dịch dài.

Choreography không dùng orchestrator; các dịch vụ tự lắng nghe sự kiện và thực thi tương ứng. Đặc điểm của mô hình này là:

- Không có bộ điều phối trung tâm[3].
- Mở rộng tự nhiên theo kiến trúc phân tán.
- Khó kiểm soát khi quy trình phức tạp[3].

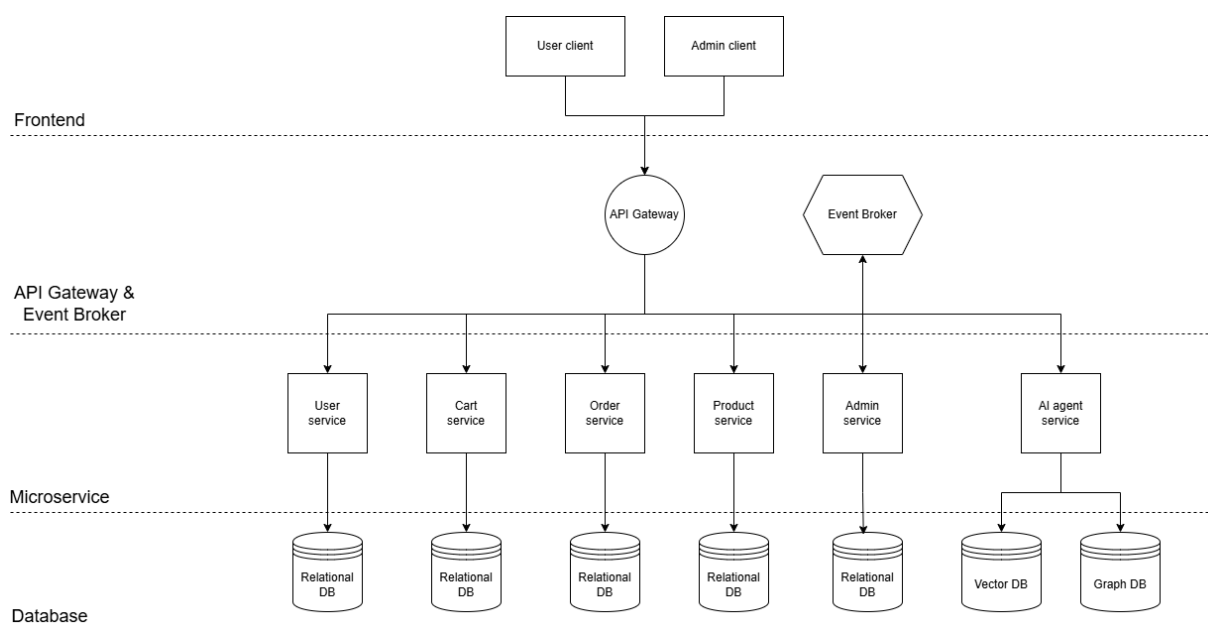
Choreography phù hợp với các quy trình nhẹ, mang tính linh hoạt cao và không yêu cầu sự điều phối tập trung. Mỗi dịch vụ phản ứng độc lập theo sự kiện, giúp hệ thống giảm bớt ràng buộc, dễ mở rộng và hoạt động theo hướng phi tập trung. Mô hình này đặc biệt hữu ích trong những luồng nghiệp vụ đơn giản hoặc các tác vụ hỗ trợ trong kiến trúc sự kiện.

Trong nhóm các mô hình điều phối nghiệp vụ, sau khi xem xét Saga và Choreography, nhóm quyết định lựa chọn Saga làm mô hình điều phối hệ thống. Quy trình đặt hàng trong thương mại điện tử là quy trình phức tạp, bao gồm nhiều bước phụ thuộc lẫn nhau như kiểm tra tồn kho, xử lý thanh toán, tạo đơn hàng, cập nhật trạng thái và gửi thông báo. Mỗi bước đều có khả năng phát sinh lỗi và yêu cầu cơ chế rollback phù hợp để đảm bảo tính nhất quán của toàn hệ thống.

Saga Orchestration có một orchestrator trung tâm chịu trách nhiệm điều phối từng bước của quy trình, theo dõi trạng thái và quyết định thực hiện hành động bù trừ (compensation) khi xảy ra lỗi. Mô hình này giúp kiểm soát luồng nghiệp vụ rõ ràng, dễ quan sát, dễ debug và phù hợp với yêu cầu thực tế của đề tài. So với Choreography, Saga tránh được tình trạng “event chaos” khi có nhiều dịch vụ phản ứng lẫn nhau, đồng thời mang lại sự minh bạch và khả năng kiểm soát cao hơn.

Vì các lý do trên, nhóm lựa chọn Saga Orchestration làm mô hình điều phối cho toàn bộ các quy trình nghiệp vụ quan trọng trong hệ thống, đặc biệt là quy trình đặt hàng.

6.1.3. Các tầng của hệ thống



Hình 6: Sơ đồ kiến trúc hệ thống

Hệ thống của nhóm được tổ chức bao gồm bốn tầng chính:

- Frontend
- API Gateway & Event Broker
- Microservice
- Database

Mỗi tầng đảm nhiệm một vai trò riêng biệt trong việc xử lý yêu cầu, điều phối hoạt động và lưu trữ dữ liệu.

Tầng Frontend

Tầng Frontend đóng vai trò là điểm tương tác đầu tiên giữa người dùng với hệ thống. Tầng này bao gồm hai thành phần chính: Giao diện dành cho khách hàng (User client) và giao diện dành cho quản trị viên của hệ thống (Admin client). User client là nơi diễn ra toàn bộ các hoạt động của khách hàng như xem sản phẩm, thêm vào giỏ hàng, đặt hàng hoặc theo dõi tình trạng đơn hàng. Trong khi đó, Admin client phục vụ quản trị viên trong việc quản lý danh mục sản phẩm, kiểm soát đơn hàng, điều chỉnh thông tin và xử lý các thao tác vận hành nội bộ.

Cả hai giao diện đều giao tiếp với các phần khác của hệ thống thông qua một thành phần duy nhất là API Gateway, đảm bảo toàn bộ yêu cầu được thống nhất về đường đi và cách xử lý.

Tầng API Gateway và Event Broker

Tầng này bao gồm hai thành phần chính: API Gateway và Event Broker, mỗi thành phần đảm nhiệm một vai trò khác nhau trong cơ chế vận hành của hệ thống.

API Gateway hoạt động như một lớp trung gian giữa Frontend và Microservices. Đây là nơi tiếp nhận tất cả các yêu cầu từ User client và Admin client, sau đó định tuyến chúng đến đúng microservice phía dưới. Việc đặt API Gateway giữa giao diện và các service giúp kiểm soát luồng dữ liệu, hợp nhất các điểm truy cập và tạo điều kiện thuận lợi cho việc áp dụng các chính sách hệ thống trong tương lai như phân quyền, kiểm soát tốc độ và ghi log.

Song song với API Gateway, hệ thống còn sử dụng một Event Broker. Event Broker đảm nhiệm việc truyền tải và phân phối các sự kiện nội bộ giữa các service theo mô hình Publish/Subscribe. Điều này cho phép các microservice xử lý bất đồng bộ, tránh việc phải gọi trực tiếp lẫn nhau và giảm thiểu sự phụ thuộc giữa các module.

Tầng Microservice

Tầng này bao gồm sáu dịch vụ độc lập theo chức năng, được thể hiện rõ trên sơ đồ:

- User service: Quản lý thông tin người dùng và các thao tác liên quan đến tài khoản.
- Cart service: Xử lý toàn bộ hoạt động liên quan đến giỏ hàng của người dùng.
- Order service: Tiếp nhận và xử lý yêu cầu liên quan đến đặt hàng và lưu đơn hàng.
- Product service: Lưu trữ và cung cấp thông tin về sản phẩm cho hệ thống.
- Admin service: Hỗ trợ các chức năng quản trị.
- AI agent service: Thực hiện các hoạt động của Trợ lý Ảo như tư vấn sản phẩm, kiểm tra thông tin, ...

Các microservice không có bất kỳ liên kết trực tiếp nào với nhau trong sơ đồ, thay vào đó toàn bộ giao tiếp đều phải thông qua Event Broker. Điều này giúp cho mỗi service được vận hành tách biệt, giúp giảm thiểu tình trạng xung đột logic, tăng tính chủ động khi triển khai và bảo trì, và bên cạnh đó còn giúp hỗ trợ nhóm phân chia công việc dễ dàng hơn trong giai đoạn hiện thực hệ thống.

Tầng Database

Tầng Database chịu trách nhiệm lưu trữ dữ liệu cho từng microservice. Mỗi microservice có cơ sở dữ liệu riêng của nó:

- User service → Lưu trữ toàn bộ thông tin người dùng và dữ liệu tài khoản liên quan.
- Cart service → Lưu dữ liệu giỏ hàng của từng người dùng, bao gồm danh sách và số lượng sản phẩm.
- Order service → Lưu thông tin đơn hàng, trạng thái xử lý và các dữ liệu liên quan đến quá trình đặt hàng.

- Product service → Lưu trữ danh sách sản phẩm, thuộc tính và thông tin mô tả sản phẩm.
- Admin service → Lưu dữ liệu phục vụ thao tác quản trị và điều chỉnh thông tin hệ thống.
- AI agent service → Lưu trữ các vector embedding dùng cho xử lý truy vấn và gợi ý của Trợ lý Ảo. Bên cạnh đó, lưu các quan hệ dạng đồ thị giữa sản phẩm, hành vi và dữ liệu cần phân tích cho Trợ lý Ảo.

Các service chính như User, Cart, Order, Product và Admin đều sử dụng Cơ sở dữ liệu quan hệ (Relational Database), đảm bảo lưu trữ dữ liệu theo cấu trúc bảng truyền thống.

Riêng AI agent service sở hữu hai cơ sở dữ liệu độc lập: Cơ sở dữ liệu Vector (Vector Database) và Cơ sở dữ liệu Graph (Graph Database), tương ứng với hai loại dữ liệu mà service này cần quản lý. Điều này cho phép dịch vụ Trợ lý Ảo vận hành trên những tập dữ liệu phù hợp với nhu cầu xử lý của nó.

7 Kết luận và hướng phát triển

Trong chương này, nhóm ...

Tài liệu

- [1] GeeksforGeeks, “Monolithic architecture - system design,” <https://www.geeksforgeeks.org/system-design/monolithic-architecture-system-design/>, 2025, [Online]. Truy cập vào: 09-11-2025.
- [2] Microsoft, “Microservices architecture style,” <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>, 2025, [Online]. Truy cập vào: 09-11-2025.
- [3] GeeksforGeeks, “Event-driven architecture patterns in cloud native applications,” <https://www.geeksforgeeks.org/system-design/event-driven-architecture-patterns-in-cloud-native-applications/>, 2025, [Online]. Truy cập vào: 09-11-2025.
- [4] Wikipedia, “Artificial intelligence,” https://en.wikipedia.org/wiki/Artificial_intelligence, 2025, [Online]. Truy cập vào: 09-11-2025.
- [5] Google, “What is an ai agent?” <https://cloud.google.com/discover/what-are-ai-agents>, [Online]. Truy cập vào: 09-11-2025.
- [6] Amazon, “What is rag (retrieval-augmented generation)?” <https://aws.amazon.com/what-is/retrieval-augmented-generation/>, [Online]. Truy cập vào: 09-11-2025.
- [7] IBM, “What is a knowledge graph?” <https://www.ibm.com/think/topics/knowledge-graph>, [Online]. Truy cập vào: 09-11-2025.
- [8] R. D. Caballar and C. Stryker, “What is a recommendation engine?” <https://www.ibm.com/think/topics/recommendation-engine>, [Online]. Truy cập vào: 09-11-2025.



8 Phụ lục